

DATA ANALYTICS AND FORECASTING FOR SERVICEPLUS 2.0

INTERNSHIP PROJECT REPORT

Submitted by

ABHILASHA NEGI

B.Tech in Computer Science and Engineering

Bharati Vidyapeeth's College of Engineering, New Delhi

National Informatics Centre (NIC)

Ministry of Electronics and Information Technology (MeitY)

Government of India

e-Services Delivery & Technology (eSDT) Division

Internship Duration: **8 June 2026 – 31 July 2026**

INTERNSHIP OVERVIEW

Particular	Details
Intern	Abhilasha Negi
Organization	National Informatics Centre (NIC)
Ministry	Ministry of Electronics and Information Technology (MeitY)
Division	e-Services Delivery & Technology (eSDT)
Internship Duration	8 June 2026 – 31 July 2026
Project Title	Data Analytics and Forecasting for ServicePlus 2.0
Project Area	Data Analytics, Business Intelligence, Data Engineering and Forecasting
College / University	Bharati Vidyapeeth's College of Engineering
Department	Computer Science and Engineering

NATURE OF INTERNSHIP

The internship was structured around the development of an end-to-end analytical workflow for ServicePlus 2.0. Rather than treating dashboard creation as an isolated visualization task, the work followed the complete data lifecycle: source discovery, profiling, data-quality assessment, cleaning and standardisation, ETL, dimensional modelling, warehouse publication, business intelligence, forecasting, and application-level drill-down.

The project therefore combined data engineering and analytical activities. The analytical warehouse provided a common foundation for reporting and forecasting, while Apache Superset and Streamlit formed the consumption layer through which operational metrics, service-demand patterns, SLA indicators, and deeper KPI-level analysis could be explored.

PREFACE

This report presents the work undertaken during my internship at the National Informatics Centre (NIC), Ministry of Electronics and Information Technology (MeitY), Government of India, from 8 June 2026 to 31 July 2026.

The internship provided practical exposure to data engineering, data analytics, business intelligence, and forecasting in the context of a large-scale government e-Governance platform. The project focused on ServicePlus 2.0 and addressed the transformation of raw transactional service-request data into structured analytical information and decision-support outputs.

The report is organised into nine project phases. The first phase establishes the institutional context, problem statement, objectives, and scope. The second phase documents the architecture and process flow. The third phase records the source-data investigation and profiling findings that informed subsequent transformations. The fourth phase describes the analytical warehouse and Star Schema. The fifth phase documents the ETL pipeline and data-quality controls.

The later phases describe the Apache Superset dashboard suite, forecasting methodology, model evaluation, Streamlit drill-down integration, project outcomes, lessons learned, limitations, and future scope. The appendices provide quick-reference technical material such as the data dictionary and forecast model card.

Particular emphasis has been placed on documenting not only what was implemented but also why the analytical workflow was structured in this manner. The intention is to make the report useful as a technical record of the internship work as well as an academic submission.

ABSTRACT

ServicePlus 2.0 is NIC's next-generation e-Governance service delivery platform, used by State and Central Government departments to design, deploy, and track online citizen services. The platform produces high-volume operational data covering service requests, workflow events, SLA metrics, officer activities, and applicant-related attributes. Such data can provide a strong foundation for operational reporting, service-demand analysis, and predictive planning when it is organised into a reliable analytical structure.

This project developed a Data Analytics and Forecasting framework for ServicePlus 2.0 to convert raw transactional service-request data into actionable and decision-ready intelligence. The work covered source-data discovery and profiling, data-quality assessment, cleaning and standardisation, relational and dimensional modelling, ETL pipeline development, PostgreSQL data warehousing, Apache Superset dashboards, demand forecasting, and Streamlit-based drill-down analytics.

The analytical warehouse was designed around a central fact table and five conformed dimensions. The ETL workflow was responsible for extracting data, staging raw records, transforming semi-structured attributes, validating quality conditions, handling duplicates, and loading conformed records into the warehouse. Business intelligence was delivered through six dashboard modules covering overview KPIs, applications, applicants, task processing, service performance/SLA, and forecasting/trends.

Demand forecasting was addressed using SARIMA as a seasonal statistical baseline and XGBoost as a machine-learning benchmark. MAPE and RMSE were used for evaluation, with the project defining a target MAPE threshold of 15% or lower. Streamlit was integrated with Superset through URL parameters so that users could move from summary KPIs to detailed drill-down analysis while preserving state, department, service, date-range, and metric context.

Overall, the project established a structured analytical foundation for operational reporting, SLA monitoring, service-demand analysis, and predictive capacity planning.

TABLE OF CONTENTS

Phase 1: Project Overview & Organizational Scope
Phase 2: Architectural & Process Workflow Design
Phase 3: Source Data Discovery, Profiling & Cleaning Assessment
Phase 4: Relational Data Modeling & Warehouse Schema Design
Phase 5: Data Engineering & ETL Pipeline Implementation
Phase 6: Business Intelligence & Interactive Dashboards
Phase 7: Demand Forecasting & Machine Learning Modeling
Phase 8: Streamlit Integration for Drill-Down Analytics
Phase 9: Outcomes, Deliverables & Future Scope
Conclusion
References
Appendices

LIST OF FIGURES

Figure 2.1: High-Level System Architecture — Four-Layer View
Figure 2.2: Data Flow Diagram (DFD) — Level 0
Figure 2.3: Data Flow Diagram (DFD) — Level 1
Figure 4.1: ER Diagram — Application Analytics Data Warehouse (Star Schema)
Figure 5.1: ETL Pipeline — Source to Warehouse
Figure 5.2: Data Quality & Validation Checkpoints
Figure 7.1: Observed Monthly Application Volume
Figure 7.2: Reported Demand Forecast Output
Figure 7.3: Forecast Validation Framework
Figure 8.1: Superset → KPI Click → Streamlit Drill-Down Flow

LIST OF TABLES

Table 2.1: Technology Stack Summary and Selection Rationale

Table 3.1: Source OLTP Schema Inventory

Table 3.2: Initial Data Inventory and Cardinality

Table 4.1: Fact Service Transaction Data Dictionary

Table 4.2: Applicant Dimension

Table 4.3: Service Dimension

Table 4.4: Location Dimension

Table 4.5: Department Dimension

Table 4.6: Status Dimension

Table 6.1: Superset Dashboard Inventory and Purpose

Table 6.2: Key Performance Indicators

Table 7.1: SARIMA Model Parameters

Table 7.2: Model Benchmarking Matrix

Table 9.1: Deliverables Against Internship Objectives

LIST OF ABBREVIATIONS

Abbreviation	Full Form
BI	Business Intelligence
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
eSDT	e-Services Delivery & Technology
ETL	Extract, Transform, Load
KPI	Key Performance Indicator
MAPE	Mean Absolute Percentage Error
MeitY	Ministry of Electronics and Information Technology
NIC	National Informatics Centre
OLAP	Online Analytical Processing
OLTP	Online Transaction Processing
RMSE	Root Mean Squared Error
SARIMA	Seasonal Autoregressive Integrated Moving Average
SLA	Service Level Agreement
PII	Personally Identifiable Information

PHASE 1: PROJECT OVERVIEW & ORGANIZATIONAL SCOPE

1.1 Institutional Context

The National Informatics Centre (NIC), under the Ministry of Electronics and Information Technology (MeitY), Government of India, develops and operates large-scale e-Governance platforms that support public-service delivery and administrative workflows. Within NIC, the e-Services Delivery & Technology (eSDT) Division manages ServicePlus 2.0, a low-code/no-code service delivery platform deployed across 36 States and Union Territories and used for delivering more than 4,000 citizen services.

The scale of ServicePlus creates a substantial operational data footprint. Service requests, workflow events, SLA-related information, officer activities, applicant attributes, submission locations, and service metadata are generated as part of normal service-delivery operations. These records provide a useful basis for enterprise analytics, but the usefulness of such data depends on understanding its grain, completeness, consistency, and business meaning before it is aggregated or modelled.

1.2 Problem Statement

Despite the scale of the platform, reporting was primarily dependent on ad-hoc SQL-based analysis. This approach can answer individual questions but does not by itself establish a centralized analytical layer that consistently defines KPIs, supports interactive dashboards, or provides reusable inputs for forecasting. SLA violations were therefore identified reactively, while predictive demand forecasting was not available as a systematic decision-support capability.

A second challenge was the structure of the source data. The source contained both application-level information and workflow/task-level activity, and important attributes were embedded inside JSON payloads. Treating every source row as a separate application would therefore distort application counts and downstream metrics. The project consequently began with a dedicated profiling and discovery stage before defining the warehouse grain.

1.3 Objectives & Deliverables

The internship objective was to build a complete analytical workflow rather than an isolated dashboard. The intended output was a reusable path from source transactions to validated warehouse data, BI reporting, demand forecasting, and drill-down analytics.

- Build an end-to-end analytics pipeline from raw transactional data to decision-ready intelligence.
- Design a PostgreSQL Star Schema warehouse for the service-delivery domain.
- Develop Apache Superset dashboards for operational and executive reporting.
- Implement statistical and machine-learning forecasting models for demand planning.
- Establish data-quality checks before analytical publication.

- Provide technical documentation describing the design, implementation, findings, and outcomes.

1.4 Scope of Work

The scope covered source-data discovery, data profiling, PII identification, semi-structured JSON investigation, data cleaning and standardisation, ETL design, dimensional modelling, PostgreSQL warehouse implementation, BI dashboards, SLA analysis, demand forecasting, Streamlit integration, and technical documentation. The work was deliberately structured so that the findings from each phase could inform the next phase instead of treating the phases as independent activities.

PHASE 2: ARCHITECTURAL & PROCESS WORKFLOW DESIGN

This phase traces the end-to-end execution flow of the Data Analytics and Forecasting system. It maps the data movement from ServicePlus source records through staging, transformation, quality validation, warehouse publication, forecasting, visualization, and final decision consumption. The architecture separates operational data generation from analytical processing so that reporting and modelling can work against a controlled analytical layer.

2.1 High-Level System Architecture

The high-level architecture is organized into four principal layers: ingestion, processing/profiling, analytical warehousing, and forecasting/visualization. The separation is important because each layer has a distinct responsibility. Ingestion is concerned with acquiring source data without placing analytical load on the production system. Processing is responsible for understanding and improving data quality. The warehouse provides a stable analytical representation. Forecasting and visualization then consume the conformed analytical data.

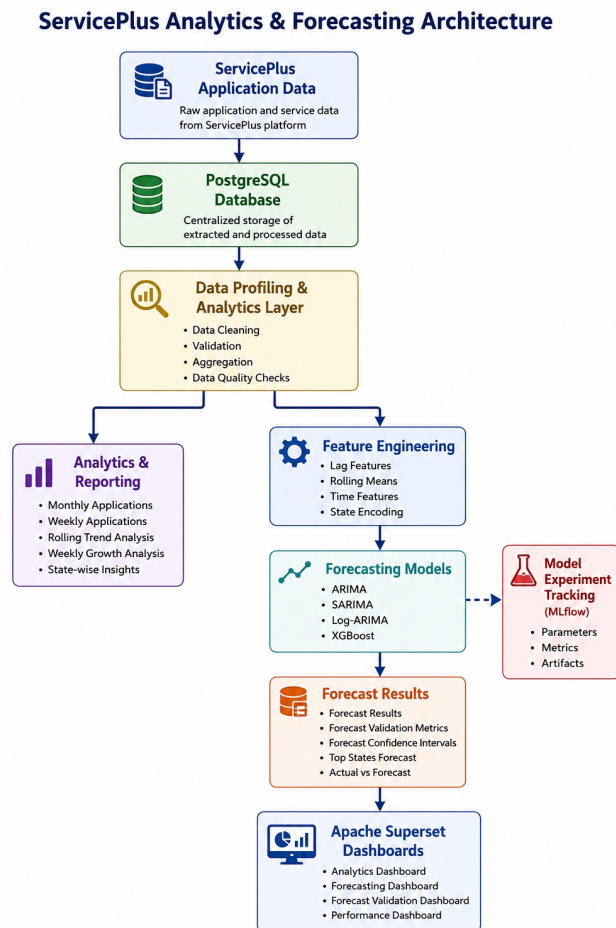


Figure 2.1: High-Level System Architecture — Four-Layer View

The architecture diagram represents the movement from raw ServicePlus application data into PostgreSQL, followed by profiling and analytics, feature engineering, forecasting, and

dashboard consumption. The architecture also reflects the project’s broader modelling and experiment workflow. In the formal model-benchmarking discussion later in this report, SARIMA and XGBoost are the principal models compared for demand forecasting.

2.2 DFD Level 0 — Context Diagram

The Level-0 DFD provides a system-wide view of how information moves between the ServicePlus database, extraction process, raw dataset, ETL pipeline, warehouse, forecasting layer, dashboard layer, and decision makers. It is intentionally less detailed than the Level-1 diagram because its purpose is to establish the major entities, processes, and data stores involved in the overall system.

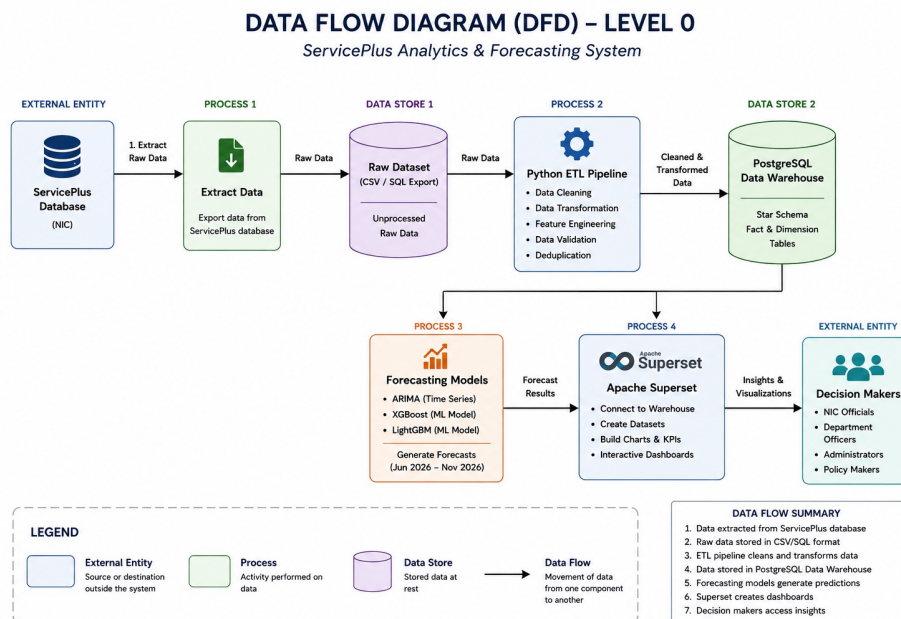


Figure 2.2: Data Flow Diagram (DFD) — Level 0

Trace Narrative

1. **Extraction:** The ServicePlus database acts as the system-of-record source. A scheduled extraction process pulls application, service, and task-level transactional records.
2. **Staging:** Extracted records are placed in a raw dataset as unprocessed CSV/SQL-style data. This staging step decouples source extraction from downstream transformations and allows the raw state of the data to be retained for audit and replay.
3. **Transformation:** The Python ETL pipeline reads the raw dataset and performs cleaning, transformation, feature engineering, validation, and deduplication.
4. **Warehouse loading:** Cleaned and conformed records are loaded into the PostgreSQL analytical warehouse using a Star Schema representation.

5. Analytical fan-out: Forecasting models consume warehouse data for demand prediction, while Apache Superset independently connects to the same warehouse to build datasets, charts, KPIs, and dashboards.
6. Decision consumption: Forecast results and dashboard insights are consumed by NIC officials, department officers, administrators, and policy makers. KPI interactions can redirect a user to a corresponding Streamlit deep-dive page using URL parameters.

2.3 DFD Level 1 — ETL & Warehouse Processes

The Level-1 DFD expands the ETL portion of the system. It makes the quality gate explicit: transformed records do not automatically enter the warehouse. They pass through validation and deduplication controls, while failed records are quarantined for investigation.

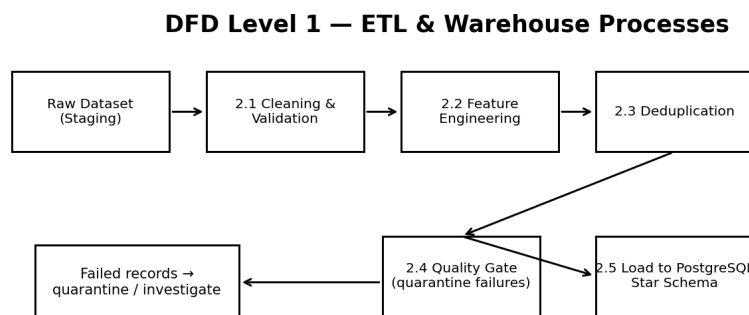


Figure 2.3: Data Flow Diagram (DFD) — Level 1: ETL and Warehouse Processes

2.4 System Architecture Layers — Detailed Responsibilities

Ingestion Layer

- Source: ServicePlus 2.0 application data, including applications, service requests, and task events.
- Input: batch exports rather than direct analytical access to the production OLTP database.
- Staging: raw, unprocessed records retained in a PostgreSQL/flat-file staging representation.
- Purpose: decouple extraction cadence from ETL execution and provide a basis for replay and audit.

Processing / Profiling Layer

- Completeness, schema-drift, duplicate, and outlier checks.
- Standardisation and null handling.
- Daily, weekly, and monthly metric roll-ups where required.

- Business-rule validation before publication.
- Quality gate that quarantines records failing required checks.

Analytical Warehouse Layer

- PostgreSQL 16+ as the warehouse platform.
- Star Schema with a central fact table and conformed dimensions.
- Analytical dimensions covering date/time context, service, department, location, and applicant information.
- Single analytical source shared by BI and forecasting workloads.

Forecasting Layer

- Lag features and rolling statistics derived from historical demand.
- Calendar/time features and state encoding.
- Seasonal statistical modelling with SARIMA.
- Feature-driven machine-learning benchmarking with XGBoost.

Visualization Layer

- Apache Superset as the primary dashboard surface.
- Warehouse-backed datasets for analytics, forecasting, and performance views.
- KPI-based redirection to Streamlit for deeper drill-down analysis.

2.5 Technology Stack Rationale

The stack is deliberately open-source and Python-centric, minimizing licensing overhead while keeping the layers interoperable through PostgreSQL. The technology choices also reflect the different responsibilities of the system: Python supports ETL and model development; PostgreSQL provides reliable relational storage and analytical querying; Superset provides BI; Statsmodels provides an interpretable SARIMA implementation; XGBoost provides a non-linear machine-learning benchmark; and Streamlit provides a rapid path from validated analytical logic to an interactive internal tool.

Technology	Version	Role	Selection Justification
Python	3.11	ETL scripting, feature engineering, model training and orchestration	Mature data-engineering and ML ecosystem including Pandas, Statsmodels and Scikit-learn; suitable for ETL-heavy workflows.

PostgreSQL	16+	Staging store and analytical Star Schema warehouse	Strong relational capabilities, window functions/CTEs, ACID guarantees and direct Superset connectivity.
Apache Superset	3.x	BI and visualization layer	Open-source dashboarding, PostgreSQL connectivity, interactive charts and role-oriented reporting.
Statsmodels	Latest stable	SARIMA implementation	Reference-grade statistical time-series implementation with interpretable model parameters.
XGBoost	Latest stable	Feature-driven demand forecasting	Captures non-linear, multivariate effects such as state, department and seasonal relationships.
Streamlit	Latest stable	KPI drill-down application	Rapid Python-based application development and direct integration with validated analytical logic.

PHASE 3: SOURCE DATA DISCOVERY, PROFILING & CLEANING ASSESSMENT

The profiling phase was treated as an independent discovery activity rather than an implicit part of ETL. The purpose was to understand the source before deciding what should become fact records, dimensions, reporting measures, or workflow-level structures. This was particularly important because the raw data contained repeated application identifiers, workflow activity, and semi-structured JSON payloads.

3.1 Source OLTP Schema Inventory

OLTP Table	Purpose	Key Attributes
service_request	Primary application transactions	application_id, service_id, applicant_id, status, dates
workflow_stage	Workflow progression	stage, officer, timestamps
approval_log	Approval history	action, approver, remarks
officer_master	Officer metadata	officer_id, department
citizen_master	Citizen profile	applicant details
district_master	Administrative geography	district, state

3.2 Source Data Discovery, Profiling & Pre-ETL Assessment

Before implementing the ETL pipeline, the source data was explored to establish its natural grain, identify semi-structured attributes, measure data-quality characteristics, and determine the transformations required for analytics. The most important analytical decision at this stage was that the source could not be interpreted as a simple one-row-per-application table.

3.2.1 Initial Data Inventory and Grain Analysis

Profiling Attribute	Finding
Unique applications	~711K
Raw rows	2,132,046
Unique application IDs	710,682
Unique application reference numbers	710,682
Unique services	35
Application statuses	4

The raw row count is larger than the application count because multiple records can be associated with the same application. Application IDs and application reference numbers were therefore examined as business keys, while workflow/task activity was treated separately.

3.2.2 Source Schema and Semi-Structured Data Investigation

The `initiated_data` and `execution_data` JSON fields were inspected directly because important analytical attributes were embedded inside semi-structured payloads. This investigation was necessary before the warehouse schema could be defined, since fields needed for service, department, location, status, submission, due-date, and workflow analysis were not all exposed as simple relational columns.

initiated_data

- Major sections: `appl_info`, `next_task_data`, `application_form_attributes`, and `enclosure_data`.
- Approximately 25 fields were identified within `appl_info`.
- Important attributes included `appl_id`, `appl_ref_no`, `service_id`, `base_service_id`, `service_name`, `department_id`, `department_name`, `submission_location`, `submission_date`, `submission_mode`, `due_date`, and `user_id`.
- Payment-related attributes were also present in the payload.

execution_data

- `task_info` exposed approximately 23 identified fields.
- Important attributes included `task_id`, `task_name`, `task_type`, `task_action`, `task_action_detail`, `received_time`, `executed_time`, officer/user details, and application identifiers.
- Payment-related fields were also observed.
- The analysis established that application-level and workflow-level information required separate analytical treatment.

3.2.3 PII and Sensitive-Field Identification

Sensitive attributes identified during source inspection included applicant name, address, date of birth, and photograph/image-path information. The purpose of identifying these fields was not to use them indiscriminately in reporting, but to distinguish PII-bearing attributes from the fields required for analytical dimensions and aggregate reporting.

3.2.4 Data-Quality Profiling

Null analysis showed that `execution_data` was populated for 282,273 profiled records, while 1,849,773 records did not contain execution data. The absence of execution data was interpreted in context because execution information is generated only after an application progresses into workflow execution. This distinction prevented a contextual absence from being treated automatically as a source defect.

Cardinality analysis established 710,682 unique application IDs, 710,682 unique application reference numbers, 35 unique services, and four application statuses. Additional cardinality checks covered departments, locations, and workflow actions.

Temporal completeness showed observed source coverage from August 2023 through May 2026, representing 34 continuous months with no missing month identified in the profiled

period. Geographic profiling highlighted higher-volume distributions including Maharashtra, Andhra Pradesh, Karnataka, Kerala, and Uttar Pradesh, while lower-volume observations included Lakshadweep and other geographically smaller service populations.

3.2.5 Outlier / Anomaly Screening

Period	Observed Volume
April 2026	292,534
October 2025	216,493
May 2026	136,997
November 2025	14,396
September 2025	13,196

These observations were treated as profiling findings to be investigated and contextualised. They were not automatically deleted as erroneous records. In an operational service-delivery dataset, unusually high or low volumes may reflect real administrative demand, changes in workflow behaviour, or other contextual effects; therefore, the profiling stage documented the observations rather than applying an unsupported deletion rule.

3.2.6 Department, Location and Workflow Analysis

Department Category	Applications
Revenue Department	319,487
UIDAI State Verification	281,187
Revenue and Disaster Management Department	39,455

Multiple naming variants were observed for the same or closely related organizational entities, including Revenue Department, DEPARTMENT OF REVENUE, UIDAI, and UIDAI State Verification. These variations motivated department-name standardisation before analytical aggregation.

Submission-location values combined organizational and geographic information, including state-level Revenue and UIDAI verification locations. This supported the later separation of department and location attributes in the analytical model.

Workflow Activity	Observed Value
Forward	182,727
Deliver	93,474
Reject	6,045
Null action detail	27

High-frequency task types included Verification State centenarians level (54,789), Verification at Sub District Level (30,504), and Send the Sub-District Level verification to UIDAI (26,082). Workflow records represented activity associated with applications rather than independent applications. Workflow actors included automated system accounts, internal user identifiers, and email-style identifiers, requiring careful treatment during workflow modelling.

3.2.7 Pre-ETL Cleaning Requirements

The profiling stage directly informed the cleaning rules applied during ETL. Department standardisation, service standardisation, application/workflow grain separation, JSON extraction, validation, and deduplication were identified as explicit requirements.

- Six observed department-name variants were consolidated into two standardized analytical names.
- Formatting variations were normalised where they represented the same service, including variants of Aadhaar Verification Portal by State - Assam.
- Application-level records were distinguished from workflow events.
- Required application and workflow attributes were identified for JSON extraction.
- Data-quality checks were defined before loading.
- Duplicate and repeated records were considered during transformation.

The resulting pre-ETL workflow was: Extract → Profile → Inspect → Define Cleaning Rules → Transform → Validate → Load. This assessment provided the evidence for the later Star Schema design and ETL transformations.

PHASE 4: RELATIONAL DATA MODELING & WAREHOUSE SCHEMA DESIGN

The analytical warehouse converts normalized OLTP transactions into a dimensional Star Schema centred on the fact_service_transaction table. The design minimizes unnecessary analytical joins, supports aggregation-oriented queries, and provides a common source for both BI and forecasting workloads.

4.1 Implemented Star Schema Entity-Relationship Diagram

The project Entity-Relationship Diagram represents the analytical warehouse as a central fact table connected to conformed dimensions. The model is organized around the natural grain of a citizen service transaction and separates descriptive context from measurable transaction attributes.

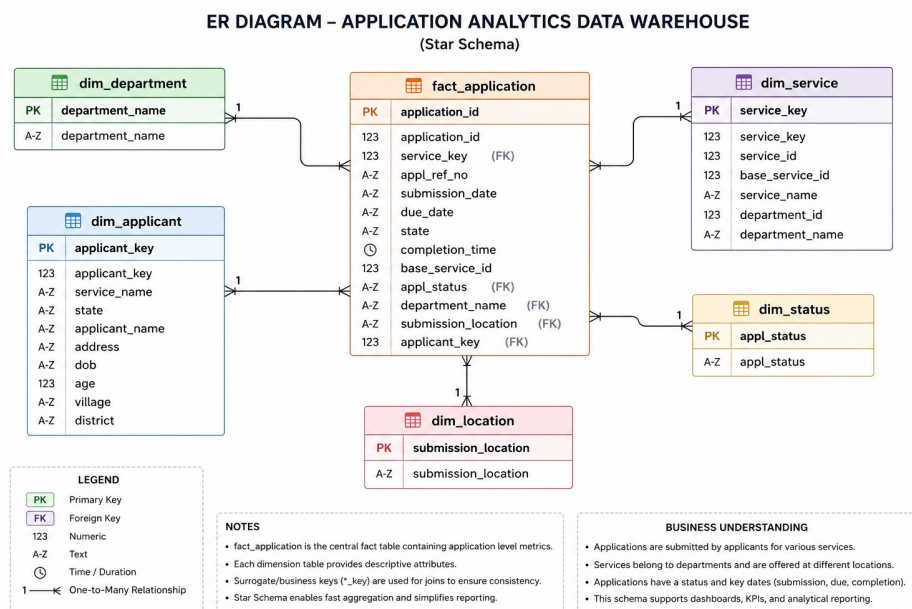


Figure 4.1: ER Diagram — Application Analytics Data Warehouse (Star Schema)

4.2 Dimensional Modeling Rationale

The star topology centres one fact table around five conformed dimensions. The design was chosen to mirror the natural grain of a citizen service transaction and to make the questions asked by dashboards explicit.

Dimension / Fact	Analytical Question Addressed
Fact service transaction	What transaction occurred, when did it occur, and what operational measures are associated with it?

Applicant	Who submitted the service request and what applicant-level context is available for analysis?
Service	What service was requested and which service identifiers describe it?
Department	Which department owns or handles the transaction?
Location	Where was the transaction submitted or associated geographically?
Status	What is the current lifecycle outcome of the transaction?

Surrogate integer keys were used to decouple the warehouse from source-system key churn and keep analytical join predicates efficient. The central fact table therefore stores foreign keys into descriptive dimensions rather than repeatedly storing every descriptive attribute.

4.3 Data Dictionary — fact_service_transaction

Column	Data Type	Key	Description
application_id	BIGINT	PK	Unique surrogate identifier for the transaction
application_ref_no	VARCHAR(50)	Unique	Citizen-facing application reference number
service_key	INT	FK → dim_service	References the requested service
applicant_key	INT	FK → dim_applicant	References the applicant
department_key	INT	FK → dim_department	References the owning department
location_key	INT	FK → dim_location	References the submission office/location
status_key	INT	FK → dim_status	References the current lifecycle status
submission_date	DATE	—	Date the application was submitted

completion_date	DATE	—	Date the application was completed/closed
processing_hours	NUMERIC	—	Total processing duration in hours
sla_status	VARCHAR(20)	—	Within SLA or Breached

The grain of the fact table is one record per application/service-request transaction. Establishing this grain is important because it prevents workflow events from being mistaken for independent service requests.

4.4 Dimension Tables

Table 4.2: dim_applicant

Column	Data Type	Key	Description
applicant_key	INT	PK (surrogate)	Surrogate key
applicant_name	VARCHAR(150)	—	Applicant name
state	VARCHAR(60)	—	State of residence
district	VARCHAR(80)	—	District of residence
age	INT	—	Applicant age
gender	VARCHAR(15)	—	Gender

Table 4.3: dim_service

Column	Data Type	Key	Description
service_key	INT	PK (surrogate)	Surrogate key
service_id	VARCHAR(30)	—	Service identifier
service_name	VARCHAR(200)	—	Service name
base_service_id	VARCHAR(30)	—	Parent/base service reference
department_name	VARCHAR(150)	—	Owning department (denormalized)

Table 4.4: dim_location

Column	Data Type	Key	Description
location_key	INT	PK (surrogate)	Surrogate key

submission_location	VARCHAR(120)	—	Office / district of submission
state	VARCHAR(60)	—	State
district	VARCHAR(80)	—	District

Table 4.5: dim_department

Column	Data Type	Key	Description
department_key	INT	PK (surrogate)	Surrogate key
department_name	VARCHAR(150)	—	Government department name

Table 4.6: dim_status

Column	Data Type	Key	Description
status_key	INT	PK (surrogate)	Surrogate key
appl_status	VARCHAR(50)	—	Pending / Delivered / Rejected / Forwarded

4.5 Reference Query — Star Join

A representative analytical query joins the central fact table to service, department, status, and location dimensions. This illustrates how the Star Schema supports readable analytical queries without requiring the dashboard layer to understand the original JSON payload structure.

```

SELECT
    f.application_id,
    s.service_name,
    d.department_name,
    st.appl_status,
    l.state
FROM fact_service_transaction f
JOIN dim_service s
    ON f.service_key = s.service_key
JOIN dim_department d
    ON f.department_key = d.department_key
JOIN dim_status st
    ON f.status_key = st.status_key
JOIN dim_location l
    ON f.location_key = l.location_key;

```

PHASE 5: DATA ENGINEERING & ETL PIPELINE IMPLEMENTATION

The ETL phase operationalized the findings from profiling and translated them into a controlled source-to-warehouse workflow. The pipeline was designed so that extraction, transformation, validation, deduplication, and publication could be reviewed independently.

5.1 Automated Extraction & Profiling

A read-only PostgreSQL replica was used as the extraction source. Python, using SQLAlchemy and Pandas, executed scheduled batch extraction jobs. The profiling pipeline computed null ratios, duplicate counts, cardinality statistics, and temporal completeness. Invalid records were quarantined before warehouse loading, and profiling reports were generated for audit and quality review.

5.2 ETL Pipeline Workflow

The implementation follows a controlled source-to-warehouse sequence. Each stage has a defined responsibility and contributes to the quality of the final analytical dataset.

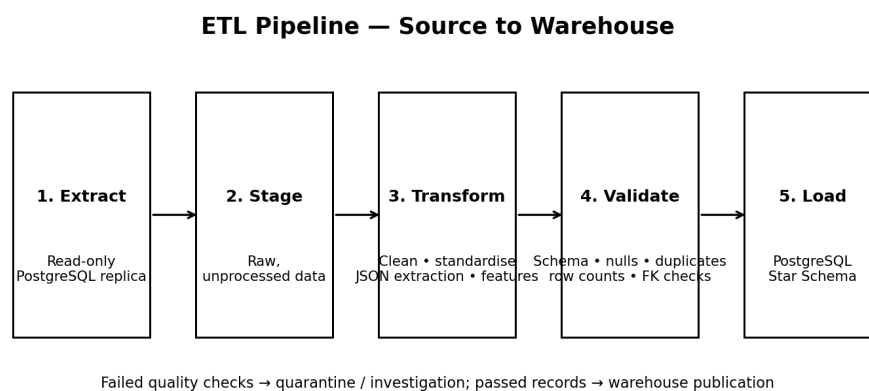


Figure 5.1: ETL Pipeline — Source-to-Warehouse Data Movement

1. Extraction — read source data from the read-only PostgreSQL replica.
2. Staging — land extracted records as raw, unprocessed data.
3. Transformation — clean, standardise, extract JSON attributes, and prepare analytical fields.
4. Validation — apply schema, null, duplicate, row-count, and referential-integrity checks.
5. Deduplication — handle repeated records before publication.
6. Loading — load validated records into the PostgreSQL Star Schema.
7. Audit / quality checks — record validation outcomes and quarantine failed records where required.

5.3 Data Quality Validation

Data quality is enforced using lightweight Python-based checks executed prior to warehouse loading. The purpose is to prevent invalid or structurally inconsistent records from becoming part of the analytical source of truth.

- Schema consistency
- Null-ratio thresholds
- Duplicate limits
- Referential integrity
- Row-count reconciliation

Column-level data quality checks

```
assert df.isnull().sum().sum() == 0
assert df.duplicated().sum() == 0
assert null_ratio < 0.05
assert duplicate_rows == 0
assert foreign_key_errors == 0
```

5.4 ETL Execution Model

The execution model follows Extract → Transform → Load into the PostgreSQL Star Schema. Refresh is performed incrementally using materialized views, while automated Python assertions are executed before each publication. The validation layer checks schema consistency, row counts, null thresholds, duplicate limits, and referential integrity.

Data Quality & Validation Checkpoints

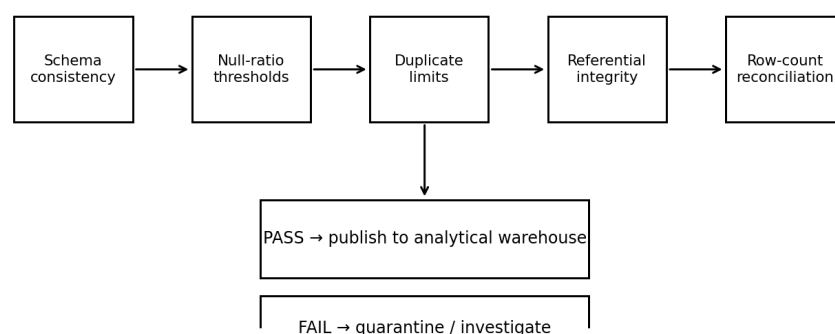


Figure 5.2: Data Quality & Validation Checkpoints within the ETL Pipeline

PHASE 6: BUSINESS INTELLIGENCE & INTERACTIVE DASHBOARDS

The ServicePlus Analytics Dashboard suite consists of six integrated modules built on Apache Superset over the PostgreSQL analytical warehouse. The dashboard layer is designed as the primary consumption surface for operational reporting, KPI monitoring, demand analysis, service performance, and forecast interpretation.

6.1 Dashboard Architecture and Implemented Overview

The dashboard suite provides interactive filtering, drill-down analysis, and role-oriented reporting. Because the dashboards query a conformed warehouse rather than the raw transactional payload, the same analytical definitions can be reused across multiple visualizations.

6.2 Dashboard Module Inventory

Dashboard Module	Key Metrics / Visualizations
Module 1 — Overview	711K total applications; 668K in-progress; 42.7K processed; 35 active states; 35 services; state-wise distribution; status trendline.
Module 2 — Applications	Department-wise analysis; top requested services; donut charts; monthly demand trend highlighting a November 2025 spike.
Module 3 — Applicants	3.33K unique applicants; 158 districts covered; state-wise applicant totals; age distribution and demographic analysis.
Module 4 — Task Processing	533K tasks; 119K completed; 9.38K rejected; workflow-stage outcomes; officer throughput monitoring.
Module 5 — Service Performance / SLA	70.48% SLA compliance; 376K on-time; 157K breached; average completion time 1.27K hours; office breach leaderboards.
Module 6 — Forecasting & Trends	Next-month expected applications: 371K; MoM change: -5.19%; service-demand heatmap; forecast trend visualization.

6.3 Implemented Dashboard Views

Applications and Demand Analysis

The applications view is intended to examine volume and demand patterns. Department-wise analysis, top-requested services, application-status distribution, and monthly demand trends are combined to give an operational picture of where service activity is concentrated and how demand changes over time. The November 2025 spike is highlighted as an example of a demand pattern that should be interpreted rather than ignored.

Applicant and Geographic Analysis

The applicant and geographic views examine state-wise applicant totals, district coverage, age distribution, and demographic summaries. These views support the interpretation of service demand in geographic context and help administrators understand whether observed application volumes are concentrated in a limited set of locations.

Service Performance and SLA Compliance

The service-performance layer focuses on SLA compliance, turnaround-time indicators, processing-time distributions, and office-level comparisons. The purpose is to move from a single overall compliance percentage to a more actionable understanding of where service-level performance differs across the analytical dimensions.

Forecasting and Service-Demand Trends

The forecasting view connects the predictive work described in Phase 7 with the BI consumption layer. It surfaces forecast trends alongside recent application behaviour and service-demand patterns so that users can compare expected demand with the recent trajectory.

6.4 Key Performance Indicators

KPI	Observed Value	Interpretation / Note
Total Applications	711K	Across all states and services
In-Progress Applications	668K	Active pipeline volume
Processed Applications	42.7K	Completed to date
Active States/UTs	35	Operational coverage
Districts Covered	158	Applicant geographic spread
Total Tasks	533K	Workflow-level task volume
Tasks Completed	119K	Completed workflow activity
Tasks Rejected	9.38K	Rejected workflow activity
SLA Compliance Rate	70.48%	376K on-time / 157K breached
Average Completion Time	1.27K hours	Across all services
Next-Month Forecast	371K applications	Reported MoM change: -5.19%

6.5 Common Dashboard Features

- Filters: state, district, department, and service.
- Analysis: drill-down from summary to detailed analytical views.
- Data source: PostgreSQL-backed datasets with declarative chart definitions.
- Operations: scheduled ETL-driven analytical refresh.
- Export: operational reporting support through CSV, image, and PDF output.

PHASE 7: DEMAND FORECASTING & MACHINE LEARNING MODELING

The forecasting component was designed to estimate future state-level daily and weekly application volumes over a 4–24 week horizon covering June–November 2026. The purpose was not simply to generate a number for the next period, but to provide a quantitative basis for proactive capacity planning, workload balancing, SLA monitoring, and decision support.

7.1 Forecasting Formulation

The forecasting target is represented as a future demand value derived from historical service-request behaviour and engineered temporal and operational features. The modelling workflow uses lagged demand, rolling statistics, calendar encodings, and state indicators as inputs.

Forecast objectives were defined as follows:

- Predict application demand for proactive capacity planning.
- Support workload balancing and SLA monitoring.
- Generate forecast intervals for decision support.

7.2 SARIMA Model Specification

SARIMA was used as the baseline seasonal forecasting model because service-request volumes can contain recurring temporal patterns. The model is expressed using non-seasonal and seasonal autoregressive, differencing, and moving-average components.

The Word-friendly formulation is:

$$\phi(B) \Phi(B^s) (1 - B)^d (1 - B^s)^D y_t = \theta(B) \Theta(B^s) \epsilon_t$$

Here, p , d , and q represent the non-seasonal autoregressive, differencing, and moving-average orders. P , D , and Q represent the corresponding seasonal orders, s is the seasonal period, and ϵ_t represents the white-noise error term. For daily data with weekly seasonality, $s = 7$; monthly seasonality can be represented with a period of 12 where applicable.

Parameter	Symbol	Selection Method
Non-seasonal AR order	p	Grid search / ACF-PACF analysis
Non-seasonal differencing	d	ADF stationarity test
Non-seasonal MA order	q	Grid search / ACF-PACF analysis
Seasonal AR order	P	Grid search over seasonal lags
Seasonal differencing	D	Seasonal stationarity test
Seasonal MA order	Q	Grid search over seasonal lags

Seasonal period	s	Fixed at 7 (weekly) or 12 (monthly)
-----------------	---	-------------------------------------

7.3 Evaluation Metrics

Two evaluation measures were used to assess forecast performance. MAPE expresses average absolute prediction error relative to actual demand and was assigned a target threshold of 15% or lower. RMSE measures the square root of mean squared error and therefore places greater emphasis on larger errors.

$$\text{MAPE} = (100 / n) \sum |(y_t - \hat{y}_t) / y_t|$$

$$\text{RMSE} = \sqrt{[(1 / n) \sum (y_t - \hat{y}_t)^2]}$$

- **MAPE target threshold:** $\leq 15\%$.
- **RMSE:** minimise subject to cross-validation across state-level time series.

7.4 Feature Engineering Pipeline

The feature-engineering stage converts raw chronological observations into model-ready predictors. The objective is to capture both short-term autocorrelation and broader seasonal/calendar behaviour.

- Lag variables: 7-, 14-, and 30-day lags of the target series.
- Rolling statistics: 7- and 30-day rolling mean and standard deviation.
- Calendar features: weekday, month, quarter, fiscal year, and holiday indicators.
- State categorical encoding for global cross-state models.
- Historical demand trends and seasonal indicators.

7.5 Model Benchmarking

Model	Category	Purpose
SARIMA	Statistical	Seasonal demand forecasting
XGBoost	Machine Learning	Non-linear, multivariate regression

The comparison between SARIMA and XGBoost provides two complementary modelling perspectives. SARIMA directly represents temporal dependence and seasonality, making its behaviour relatively interpretable. XGBoost can incorporate a wider feature set and model non-linear interactions, which is useful when demand is influenced by state, calendar, and other engineered predictors.

7.6 XGBoost Objective Function

For the gradient-boosted tree benchmark, the regularized objective can be written in a Word-friendly form as:

$$\text{Obj}(\theta) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

The loss term measures prediction error, while the regularization term penalizes model complexity and leaf weights. This structure helps control overfitting while retaining the ability of boosted trees to capture non-linear relationships.

7.7 Observed Demand Profile and Reported Forecast Output

The profiling stage identified substantial variation in observed monthly demand. April 2026 was the highest reported observation among the selected periods at 292,534 applications, followed by October 2025 at 216,493. The reported next-month forecast output was 371K applications, accompanied in the dashboard summary by a month-on-month change of -5.19% . The chart below uses the reported values and is intended to document the magnitude of the observations and forecast output rather than to recreate an unseen model-generated time series.

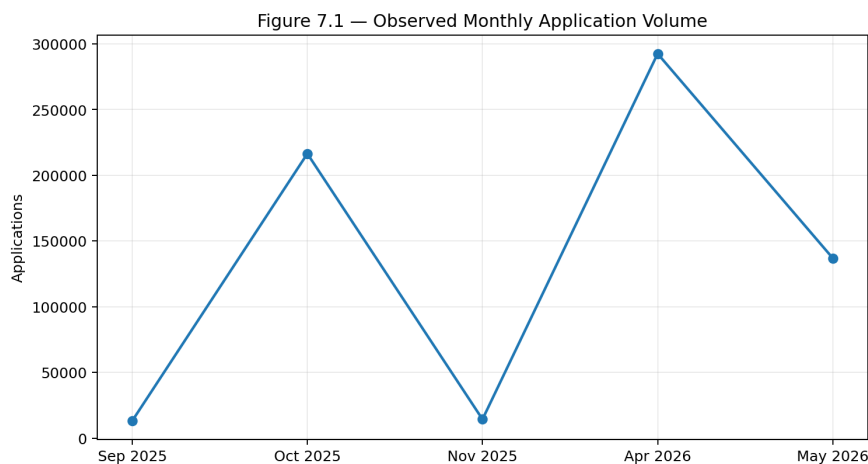


Figure 7.1: Observed Monthly Application Volume — selected reported profiling observations

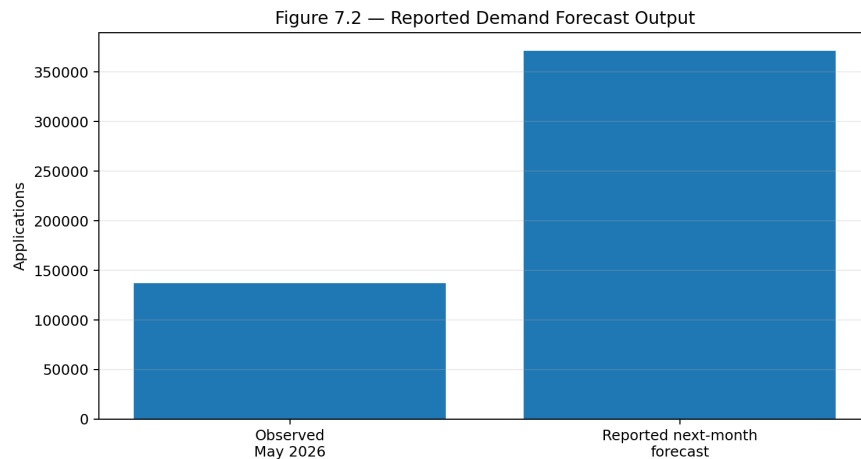
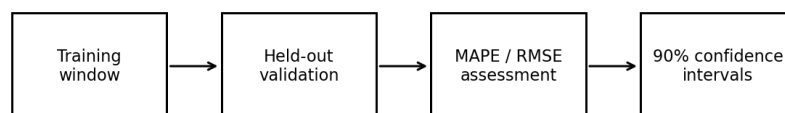


Figure 7.2: Reported Demand Forecast Output — observed May 2026 vs reported next-month forecast

7.8 Forecast Validation

Validation used a held-out validation window, MAPE and RMSE for forecast accuracy, and 90% confidence intervals around forecast estimates. The evidence considered included time-series behaviour, actual-versus-forecast comparisons, and residual diagnostics. The purpose of validation was to assess whether a model generalized beyond the data used for fitting rather than relying only on training performance.

Forecast Validation Framework



Evidence considered: time-series behaviour • actual vs forecast • residual diagnostics

Target: $\text{MAPE} \leq 15\%$ | RMSE minimised subject to cross-validation

Figure 7.3: Forecast Validation Framework

PHASE 8: STREAMLIT INTEGRATION FOR DRILL-DOWN ANALYTICS

8.1 Superset-to-Streamlit Architecture

Apache Superset is the primary dashboard surface, while Streamlit hosts deeper drill-down views for individual KPIs and metrics. The integration was designed to preserve analytical context rather than forcing users to manually reproduce their filters after moving from a summary dashboard to a detailed view.

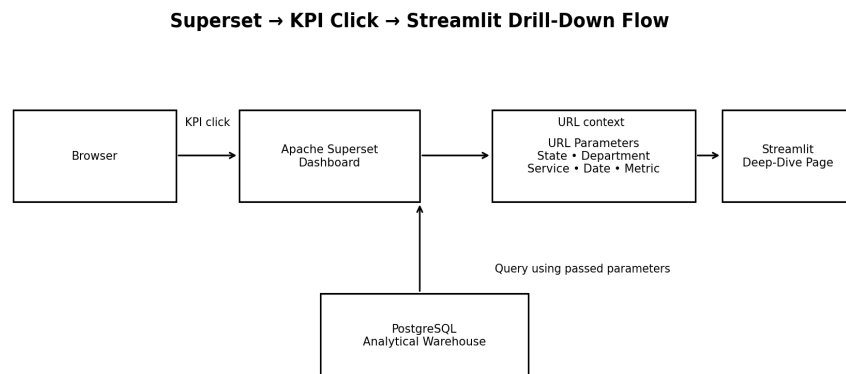


Figure 8.1: Superset → KPI Click → Streamlit Drill-Down Flow

Integration Flow

1. Dashboard surface: Superset displays KPIs across the six dashboard modules.
2. KPI interaction: a user clicks a KPI or metric element.
3. Redirect: the browser opens the corresponding Streamlit page directly.
4. Context transfer: state, department, service, date range, and metric identifier are passed as URL parameters.
5. Warehouse query: Streamlit parses the parameters and scopes queries directly against PostgreSQL.
6. Drill-down: the page renders interactive charts and, where applicable, forecast visualizations.
7. Further filtering: users can filter by state, department, service, and time range using Plotly/Matplotlib charts.

8.2 Benefits of the Integration

- Provides granular, record-level analytics beyond what a dashboard visualization alone can convey.
- Preserves filter context across the transition from Superset to Streamlit, avoiding repeated manual filtering.
- Keeps the architecture simple: Superset, Streamlit, and PostgreSQL form the core integration path.

- Allows the same analytical warehouse to serve both summary reporting and detailed exploration.

PHASE 9: OUTCOMES, DELIVERABLES & FUTURE SCOPE

9.1 Project Outcomes

The project established an analytical workflow that connected source-data understanding with operational reporting and predictive analysis. The outcome was not limited to dashboard visuals; the work created a structured warehouse layer, documented quality controls, defined forecasting inputs and evaluation criteria, and connected high-level KPIs to detailed Streamlit analysis.

- Replaced reactive, ad-hoc SQL reporting with centralized, interactive dashboards.
- Enabled proactive SLA-compliance monitoring across departments.
- Delivered state-wise demand forecasting to support predictive capacity planning.
- Established a single PostgreSQL analytical warehouse serving both BI and forecasting workloads.
- Improved the basis for evidence-based decision-making through consolidated KPIs and drill-down analytics.

9.2 Internship Deliverables

Deliverable	Status	Mapped Phase
D1 — Data Profiling Report	Delivered	Phase 3
D2 — Star Schema Design	Delivered	Phase 4
D3 — ETL Pipeline	Delivered	Phase 5
D4 — SLA Compliance Dashboard	Delivered	Phase 6
D5 — Service Demand Heatmap	Delivered	Phase 6 / Phase 7
D6 — Forecast Model & Model Card	Delivered	Phase 7
D7 — Streamlit Integration	Delivered	Phase 8
D8 — Final Report & Documentation	Delivered	Phase 9

9.3 Lessons Learned

Designing the Star Schema around the natural grain of a citizen service transaction simplified both dashboard construction and forecasting feature engineering. The decision to separate application-level transactions from workflow-level activity reduced the risk of double-counting service requests.

Enforcing data-quality checks early in the ETL pipeline reduced downstream data-cleaning effort across both BI and forecasting. The profiling stage also demonstrated that not every

missing value represents a defect; for example, absent `execution_data` can reflect an application that has not progressed into workflow execution.

The direct URL-parameter-based handoff between Superset and Streamlit proved sufficient to deliver drill-down analytics without introducing additional infrastructure. Seasonal patterns in citizen service demand, including the November 2025 spike observed in the dashboard analysis, reinforced the value of using SARIMA as a seasonal baseline alongside a feature-driven model such as XGBoost.

9.4 Challenges and Limitations

- High-volume transactional data and repeated records per application.
- Semi-structured JSON fields requiring explicit extraction and interpretation.
- Missing execution data for applications that had not progressed to workflow execution.
- Inconsistent department naming requiring standardisation.
- Need to distinguish application-level grain from workflow-level activity.
- Need for quality validation before analytical publication.
- Forecast uncertainty over longer horizons and dependence on historical patterns.

9.5 Future Scope

- Extend forecasting coverage to additional services and departments beyond the current pilot scope.
- Explore anomaly-detection techniques to flag abnormal application spikes for early administrative review.
- Introduce automated SLA-breach notifications to alert relevant departments proactively.
- Periodically retrain forecasting models as new transactional data becomes available.
- Expand predictive analytics for workload and capacity planning.
- Improve monitoring for data drift and model performance.

CONCLUSION

The internship project developed an end-to-end analytical framework for ServicePlus 2.0, covering the complete path from raw transactional data to business intelligence and forecasting. The work began with source-data discovery and profiling to understand the structure, natural grain, completeness, cardinality, PII, anomalies, and workflow characteristics of the source.

The profiling findings directly informed the cleaning and transformation requirements. In particular, the distinction between application-level transactions and workflow activity became a central modelling decision. Semi-structured JSON fields were investigated to identify the attributes required for analytical dimensions, while department and service naming variations were identified as standardisation requirements.

A dimensional PostgreSQL warehouse was then designed using a Star Schema with a central service-transaction fact table and five conformed dimensions. The ETL workflow provided extraction, staging, transformation, validation, deduplication, loading, and audit controls. This created a common analytical foundation for the BI and forecasting layers.

The resulting warehouse supported Apache Superset dashboards covering application demand, applicants, workflow processing, SLA performance, and forecasting/trends. SARIMA and XGBoost provided complementary approaches to demand forecasting, while MAPE and RMSE supplied quantitative evaluation criteria. Streamlit was subsequently integrated with Superset so that users could move from summary KPIs to detailed analytics while preserving analytical context through URL parameters.

Overall, the internship provided practical exposure to the complete data analytics lifecycle: source-data investigation, quality assessment, ETL, dimensional modelling, warehouse implementation, BI development, forecasting, and interactive application integration. The resulting framework provides a structured basis for more proactive, data-driven monitoring and planning within the ServicePlus 2.0 environment.

REFERENCES

1. National Informatics Centre (NIC), ServicePlus 2.0 Platform Documentation, Ministry of Electronics and Information Technology (MeitY), Government of India.
2. Hyndman, R. J. & Athanasopoulos, G., Forecasting: Principles and Practice, 3rd Edition — SARIMA methodology reference.
3. Chen, T. & Guestrin, C., “XGBoost: A Scalable Tree Boosting System,” KDD 2016 — objective function and regularization formulation.
4. Apache Superset Documentation — dashboard and dataset configuration reference.
5. PostgreSQL 16 Official Documentation — relational database, analytical querying, and materialized-view implementation reference.
6. Streamlit Documentation — interactive application development reference.

APPENDICES

APPENDIX A — DATA DICTIONARY (QUICK REFERENCE)

The following consolidated table provides a quick-reference view of the central fact table used by the analytical warehouse.

Column	Data Type	Key	Description
application_id	BIGINT	PK	Unique application identifier
application_ref_no	VARCHAR(50)	Unique	Citizen application reference
service_key	INT	FK	References dim_service
applicant_key	INT	FK	References dim_applicant
department_key	INT	FK	References dim_department
location_key	INT	FK	References dim_location
status_key	INT	FK	References dim_status
submission_date	DATE	—	Submission date
completion_date	DATE	—	Completion date
processing_hours	NUMERIC	—	Processing duration
sla_status	VARCHAR(20)	—	Within SLA / Breached

APPENDIX B — FORECAST MODEL CARD

Field	Specification
Model	SARIMA baseline with XGBoost benchmark
Objective	State-wise service-demand forecasting
Forecast Horizon	4–24 weeks; project reporting covers June–November 2026
Hyperparameters	(p,d,q)(P,D,Q,s), selected using grid-search / stationarity-informed methods
Features	Historical demand, seasonality, calendar effects, lag variables, rolling statistics, state indicators

Metrics	MAPE target $\leq$ 15%; RMSE
Assumptions	Historical seasonal patterns continue
Retraining	Periodic or after significant drift is observed
Confidence	90% confidence intervals
Consumption	Superset dashboards and Streamlit drill-down pages

APPENDIX C — REPRESENTATIVE STAR-SCHEMA QUERY

```

SELECT
    f.application_id,
    s.service_name,
    d.department_name,
    st.appl_status,
    l.state
FROM fact_service_transaction f
JOIN dim_service s
    ON f.service_key = s.service_key
JOIN dim_department d
    ON f.department_key = d.department_key
JOIN dim_status st
    ON f.status_key = st.status_key
JOIN dim_location l
    ON f.location_key = l.location_key;

```

APPENDIX D — ETL VALIDATION CHECKS

```

assert df.isnull().sum().sum() == 0
assert df.duplicated().sum() == 0
assert null_ratio < 0.05
assert duplicate_rows == 0
assert foreign_key_errors == 0

```


APPENDIX E — INTERNSHIP PHASE SUMMARY

Phase	Major Work
Phase 1	Project overview and organizational scope
Phase 2	Architecture and process workflow design
Phase 3	Source discovery, profiling and cleaning assessment
Phase 4	Relational modelling and Star Schema design
Phase 5	ETL pipeline and data-quality validation
Phase 6	Apache Superset dashboards
Phase 7	SARIMA and XGBoost forecasting
Phase 8	Streamlit drill-down integration
Phase 9	Outcomes, lessons learned and future scope